SkillSwap Design Document

Author: Achyut Katiyar **Class:** CS5610 Web Development **Date:** February 2026

Project Description

SkillSwap is a peer to peer skill exchange platform for students. Every semester students want to pick up new skills but don't know who around them can help. At the same time many students have talents they would love to share but have no easy way to find interested learners. SkillSwap solves this by letting students post what they can teach and what they want to learn so others can browse and connect. It removes the awkwardness of asking around in group chats and creates a simple space for peer to peer learning. The app uses Node.js, Express, MongoDB, and vanilla JavaScript to provide a clean browsing and posting experience.

User Personas

1. Alex

Alex is a Computer Science sophomore who is great at Python but wants to learn guitar. He does not know anyone in his circle who plays guitar and wants an easy way to find someone willing to trade skills.

2. Maria

Maria is a Music major who plays guitar and wants to learn basic coding to build a personal website. She has posted in group chats before but never got a serious response.

3. Jake

Jake is a senior who speaks fluent Japanese and wants to improve his public speaking before job interviews. He is looking for a quick and focused way to find someone who can help.

User Stories

1. As a student who has a skill to offer, I want to create a post listing what I can teach and what I want to learn, so I can find someone to exchange skills with.

2. As a student looking to learn something new, I want to browse and filter posts by skill or keyword, so I can quickly find someone offering what I need.

3. As a student interested in a post, I want to send a short message to the poster through the app, so I can introduce myself and set up a meetup.

4. As a student who posted a listing, I want to view all the inquiries I received on my post, so I can pick the best match.

5. As a student who posted a listing, I want to edit or delete my post, so I can update my availability or remove it once I find a match.

6. As a student browsing the platform, I want to see how many inquiries each post has received, so I can tell which posts are still active and open.

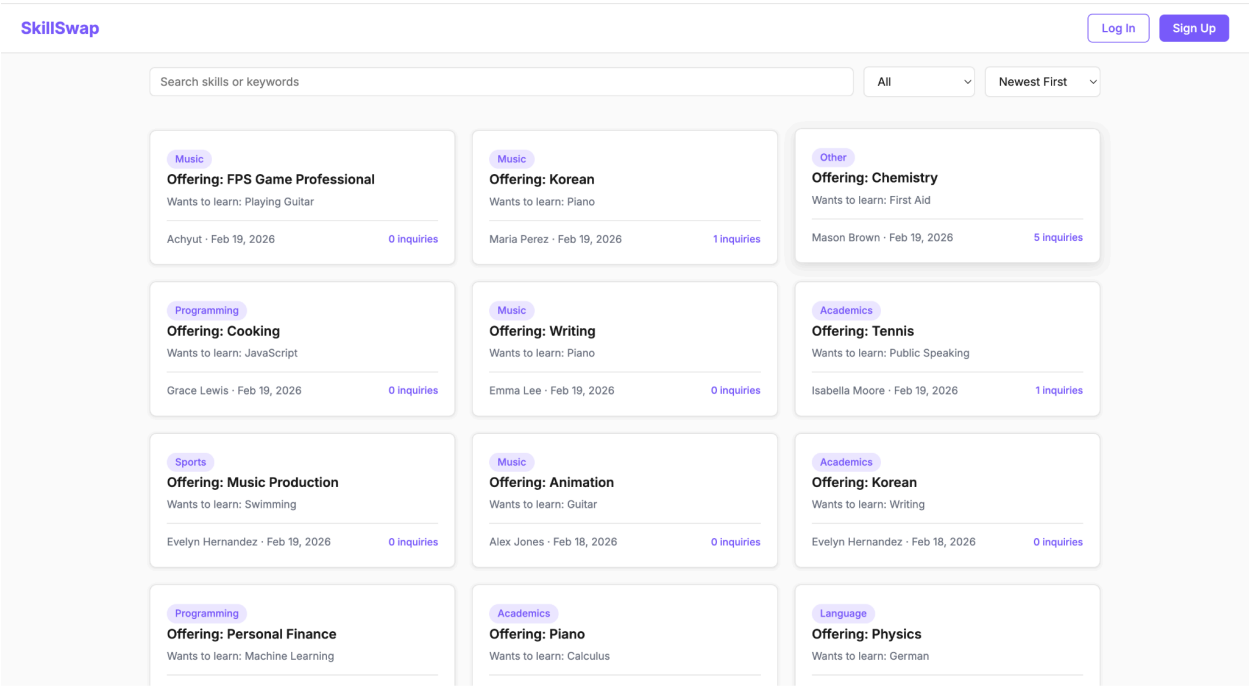
Design Mockups

[View Wireframes on Figma]

The wireframes include the following screens:

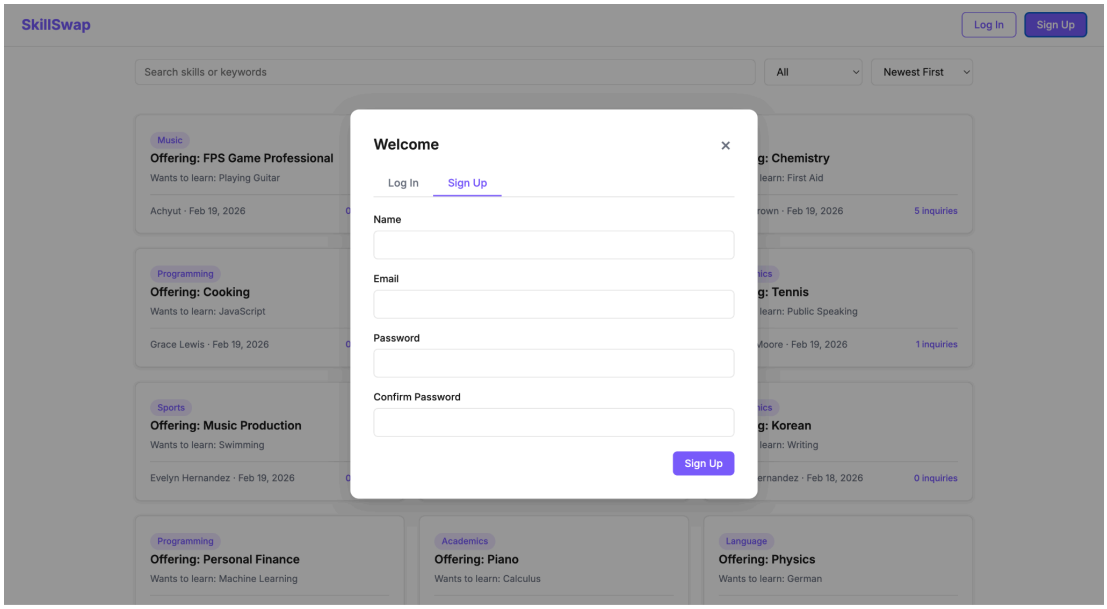
1. Home Page (Browse Posts)

The default view showing the navigation bar with SkillSwap logo, Log In and Sign Up buttons. Below the nav is a filter bar with a search input, category dropdown, and sort dropdown. The main content area displays a responsive 3 column grid of post cards. Each card shows a category badge, skill offered, skill wanted, poster name, date, and inquiry count.



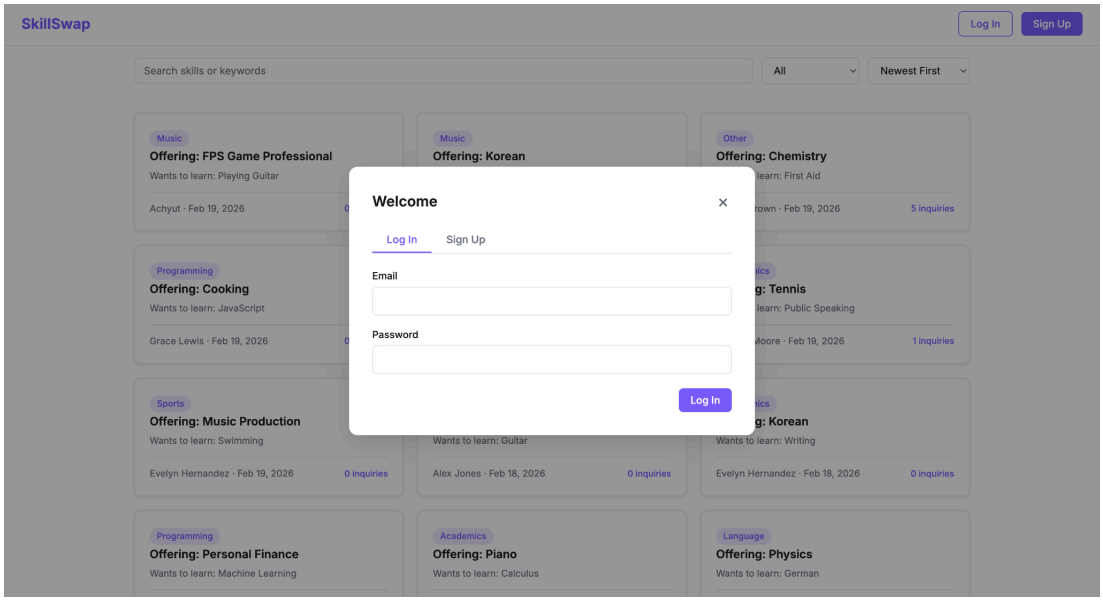
2. Sign Up Modal

A centered modal overlay with two tabs at the top for Log In and Sign Up. The Sign Up tab shows four fields: Name, Email, Password, and Confirm Password. A Sign Up button submits the form.



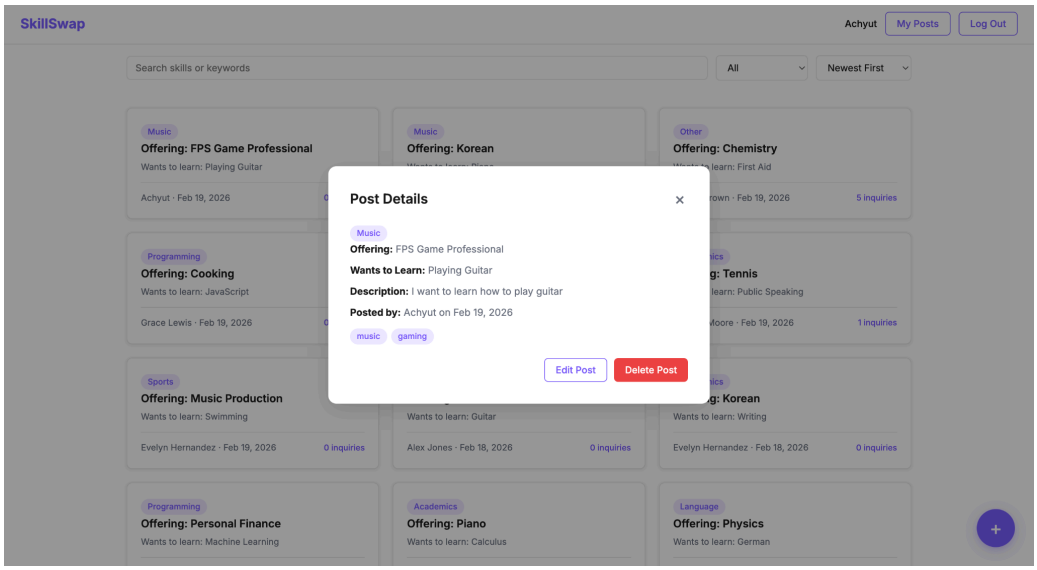
3. Log In Modal

The same modal with the Log In tab active. Shows two fields: Email and Password. A Log In button submits the form.



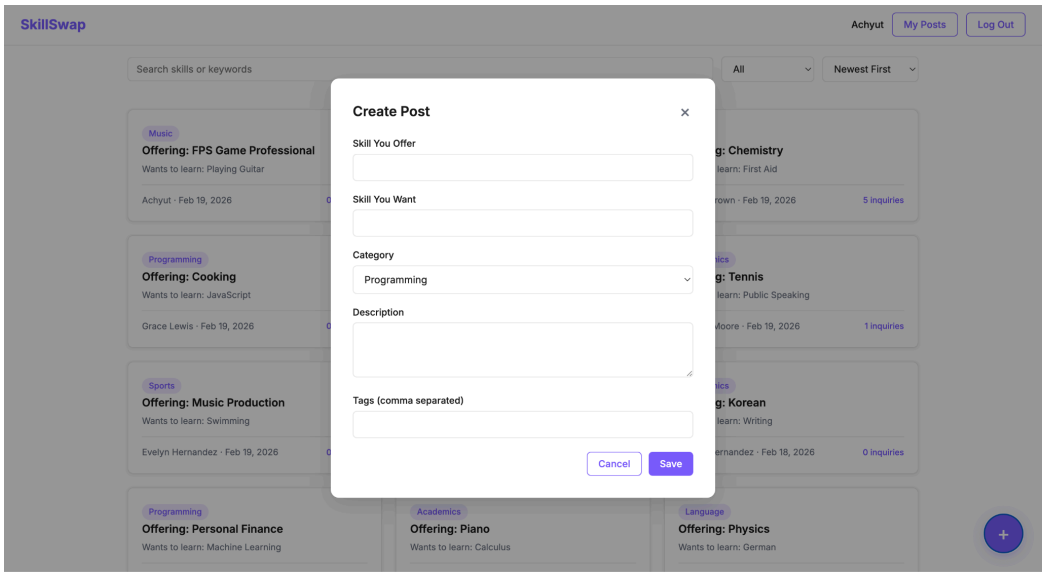
4. Post Detail Modal

A larger modal showing complete post information: category badge, skill offered, skill wanted, full description, poster name and date, and tags. Below the post info is a list of inquiry cards showing sender name, message, and date. At the bottom is an inquiry form with name, contact, and message fields for logged in users.



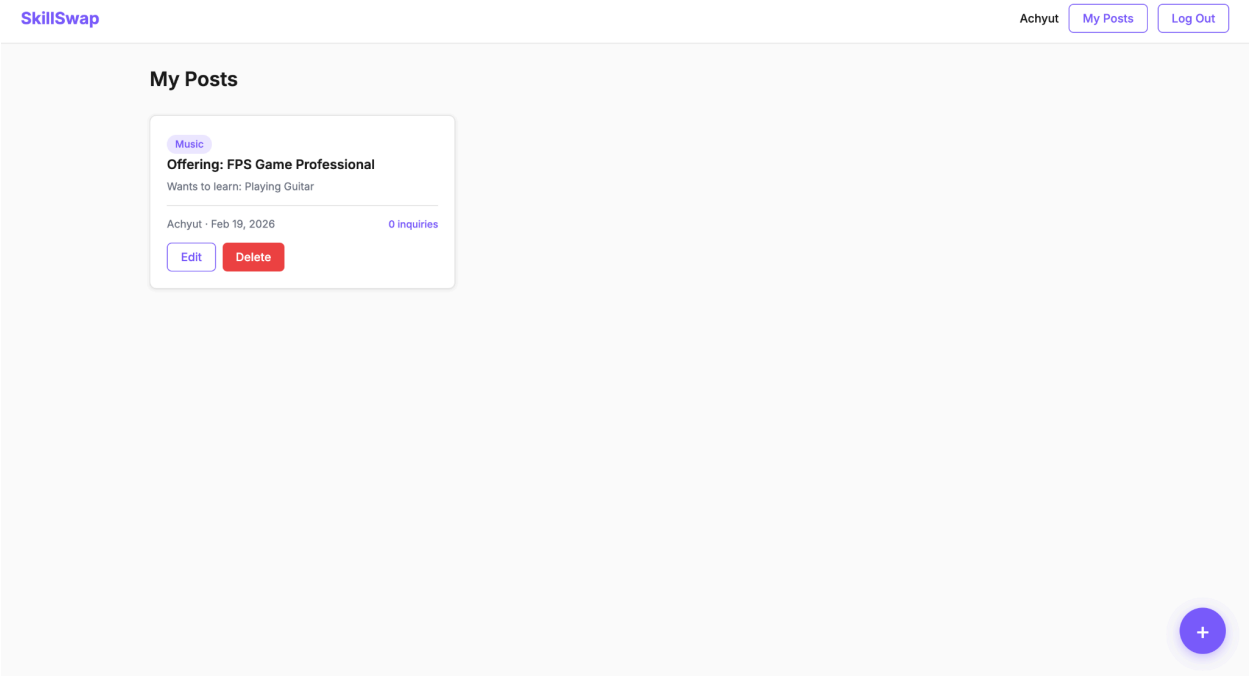
5. Create Post Modal

A form modal with fields for Skill You Offer, Skill You Want, Category dropdown, Description textarea, and Tags input. Cancel and Save buttons at the bottom.



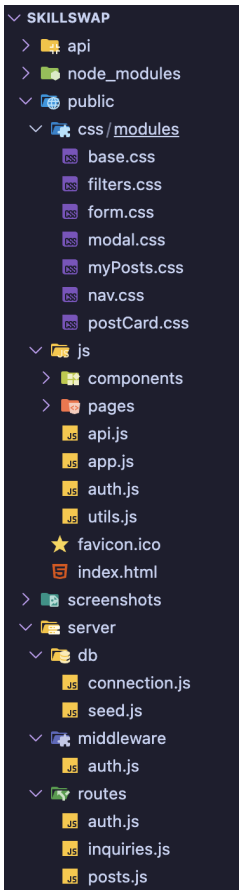
6. My Posts Page

A page showing only the logged in user's posts in a grid layout. Each card includes Edit and Delete buttons. A floating action button in the bottom right corner allows creating new posts.



Technical Decisions

Folder Architecture



Tech Stack

The backend uses Node.js with Express for routing and middleware. MongoDB was chosen as the database because the data is document oriented and the schema is flexible. The native MongoDB driver is used instead of Mongoose to keep the code simple and avoid unnecessary abstraction. The frontend is built with vanilla JavaScript, HTML5, and CSS with no frameworks or template engines.

Architecture

The app follows a single page application pattern. The server serves one HTML file and exposes a REST API. All rendering happens on the client side using JavaScript DOM manipulation. The frontend code is organized into modules where api.js handles all fetch calls, auth.js manages JWT tokens and user state, components/ contains reusable DOM element builders, and pages/ contains view rendering functions.

Authentication

JWT tokens are used for authentication. On signup or login the server returns a token that the client stores in localStorage. The token is sent as a Bearer token in the Authorization header on every authenticated request. A middleware function on the server verifies the token and attaches the user info to the request.

Database Design

Three collections are used here which are users, posts and inquiries. The poster's name is denormalized on each post to avoid extra lookups when rendering the post grid. The inquiry count is also denormalized on each post and updated with $inc on every inquiry create or delete. This avoids counting inquiries on every GET request.

SkillSwap > skillswap

Open MongoDB shell | Create collection | Refresh

Collection name	Properties	Storage size	Documents	Avg. document size	Indexes	Total index size
inquiries	-	77.82 kB	381	264.00 B	1	36.86 kB
posts	-	155.65 kB	1.1K	327.00 B	1	69.63 kB
users	-	36.86 kB	51	176.00 B	1	36.86 kB

CSS Architecture

CSS is split into modular files by component like base.css for variables and resets, nav.css for the navigation bar, postCard.css for post cards, modal.css for all modals, form.css for form elements, filters.css for the search and filter bar and myPosts.css for the my posts page and FAB button. CSS custom properties defined in base.css are used throughout for consistent theming.

Deployment

The app is configured for Vercel serverless deployment. The Express app is wrapped as a serverless function in api/index.js. Static files are served from the public/ directory. The vercel.json config routes /api/* requests to the serverless function and everything else to static files. The database is hosted on MongoDB Atlas so it is accessible from both local development and production.